

6. Hotel Room Booking System

Problem Statement

Create a hotel reservation system.

Features

- Room booking
- Check-in
- Check-out
- Room availability
- Bill generation
- Booking history

Concepts Covered

- OOP
- Constructors
- Date & Time Module
- File Handling
- Exception Handling

```
1  from datetime import datetime
2  import json
3  import os
4
5
6  class Room: 3+ usages
7      def __init__(self, room_no, price):
8          self.room_no = room_no
9          self.price = price
10         self.is_booked = False
11         self.is_checked_in = False
12
13
14  class HotelSystem: 1 usage
15
16
17      def __init__(self):
18          self.filename = "hotel_data.json"
19
20          self.rooms = {
21              101: Room(room_no=101, price=100.0),
22              102: Room(room_no=102, price=150.0),
23              103: Room(room_no=103, price=200.0)
24          }
25          self.bookings = {} # empty dictionary for active bookings
26          self.history = [] # list to keep past bookings history
27          self.next_id = 1 # ID number for the next booking
28
29          self.load_data() # loads previously saved data from the file on start
30
```

```

1  from datetime import datetime
2  import json
3  import os
4
5
6  class Room: 3+ usages
7      def __init__(self, room_no, price):
8          self.room_no = room_no
9          self.price = price
10         self.is_booked = False
11         self.is_checked_in = False
12
13
14  class HotelSystem: 1 usage
15
16
17      def __init__(self):
18          self.filename = "hotel_data.json"
19
20          self.rooms = {
21              101: Room(room_no: 101, price: 100.0),
22              102: Room(room_no: 102, price: 150.0),
23              103: Room(room_no: 103, price: 200.0)
24          }
25
26          self.bookings = {} # empty dictionary for active bookings
27          self.history = [] # list to keep past bookings history
28          self.next_id = 1 # ID number for the next booking
29
30          self.load_data() # loads previously saved data from the file on start

```

```

30
31  def show_availability(self): 1 usage
32      print("\n--- Room Availability ---")
33      for num, room in self.rooms.items():
34          status = "Booked" if room.is_booked else "Available"
35          print(f"Room {num} | Price: ${room.price} | Status: {status}")
36
37  def book_room(self): 1 usage
38      try:
39          room_no = int(input("Enter room number to book: "))
40          if room_no not in self.rooms:
41              print("Error: Room does not exist.")
42              return
43
44          room = self.rooms[room_no]
45          if room.is_booked:
46              print("Error: Room is already booked.")
47              return
48
49          name = input("Enter guest name: ")
50          in_str = input("Check-in Date (YYYY-MM-DD): ")
51          out_str = input("Check-out Date (YYYY-MM-DD): ")
52
53          # Date Module usage
54          check_in = datetime.strptime(in_str, format: "%Y-%m-%d").date()
55          check_out = datetime.strptime(out_str, format: "%Y-%m-%d").date()
56
57          if check_out <= check_in:

```

PROJECT(python) without hashtag.py x PROJECT(python) with hashtag usage.py

```
14     class HotelSystem: 1 usage
37         def book_room(self): 1 usage
54             check_in = datetime.strptime(in_str, format: "%Y-%m-%d").date()
55             check_out = datetime.strptime(out_str, format: "%Y-%m-%d").date()
56
57             if check_out <= check_in:
58                 print("Error: Check-out must be after check-in.")
59                 return
60
61             room.is_booked = True
62             days = (check_out - check_in).days
63             bill = days * room.price
64
65             self.bookings[self.next_id] = {
66                 "name": name,
67                 "room": room_no,
68                 "in": str(check_in),
69                 "out": str(check_out),
70                 "bill": bill
71             }
72             print(f"Success! Your Booking ID is: {self.next_id}")
73             self.next_id += 1
74             self.save_data()
75
76         except ValueError:
77             print("Error: Invalid date format or numbers.")
78
79     def check_in(self): 1 usage
80         try:
81             b_id = int(input("Enter Booking ID for check-in: "))
```

```
    except ValueError:
        print("Error: Invalid date format or numbers.")

def check_in(self): 1 usage
    try:
        b_id = int(input("Enter Booking ID for check-in: "))
        if b_id not in self.bookings:
            print("Error: Booking ID not found.")
            return

        b = self.bookings[b_id]
        room_no = b["room"]
        self.rooms[room_no].is_checked_in = True
        print(f"Success! Guest {b['name']} checked into Room {room_no}.")
    except ValueError:
        print("Error: Please enter a valid ID number.")

def check_out(self): 1 usage
    try:
        b_id = int(input("Enter Booking ID for check-out: "))
        if b_id not in self.bookings:
            print("Error: Booking ID not found.")
            return

        b = self.bookings[b_id]
        room = self.rooms[b["room"]]
```

```

        room.is_booked = False
        room.is_checked_in = False

        print(f"\nBILL INVOICE")
        print(f"Guest Name : {b['name']}")
        print(f"Room Number : {b['room']}")
        print(f"Check-In : {b['in']}")
        print(f"Check-Out : {b['out']}")
        print(f"Total Bill : ${b['bill']}")

        self.history.append(b)
        del self.bookings[b_id]

        self.save_data()
    except ValueError:
        print("Error: Invalid input.")

def show_history(self): 1 usage
    print("\n--- Booking History ---")
    if not self.history:
        print("No past records found.")
        return
    for h in self.history:
        print(f"Guest: {h['name']} | Room: {h['room']} | Paid: ${h['bill']} | Stay: {h['in']} to {h['out']}")

def save_data(self): 2+ usages
    data = {

```

```

93     def check_out(self): 1 usage
94         except ValueError:
95             print("Error: Invalid input.")
96
97     def show_history(self): 1 usage
98         print("\n--- Booking History ---")
99         if not self.history:
100             print("No past records found.")
101             return
102         for h in self.history:
103             print(f"Guest: {h['name']} | Room: {h['room']} | Paid: ${h['bill']} | Stay: {h['in']} to {h['out']}")
104
105     def save_data(self): 2+ usages
106         data = {
107             "next_id": self.next_id,
108             "bookings": self.bookings,
109             "history": self.history
110         }
111         with open(self.filename, "w") as f:
112             json.dump(data, f)
113
114     def load_data(self): 1 usage
115         if os.path.exists(self.filename):
116             with open(self.filename, "r") as f:
117                 data = json.load(f)
118                 self.next_id = data.get("next_id", 1)
119                 self.bookings = {int(k): v for k, v in data.get("bookings", {}).items()}
120                 self.history = data.get("history", [])
121                 for b in self.bookings.values():
122                     self.rooms[b["room"]].is_booked = True

```

```

145         self.rooms[self.room][self.is_booked] = True
146
147
148     def main(): 1 usage
149         hotel = HotelSystem()
150         while True:
151             print("\n HOTEL SYSTEM MENU ")
152             print("1. View Rooms\n2. Book Room\n3. Check-In\n4. Check-Out & Bill\n5. Booking History\n6. Exit")
153
154             try:
155                 choice = int(input("Choose an option (1-6): "))
156             except ValueError:
157                 print("Invalid input. Please enter a number from 1 to 6.")
158                 continue
159
160             if choice == 1:
161                 hotel.show_availability()
162             elif choice == 2:
163                 hotel.book_room()
164             elif choice == 3:
165                 hotel.check_in()
166             elif choice == 4:
167                 hotel.check_out()
168             elif choice == 5:

```

```

159
160             if choice == 1:
161                 hotel.show_availability()
162             elif choice == 2:
163                 hotel.book_room()
164             elif choice == 3:
165                 hotel.check_in()
166             elif choice == 4:
167                 hotel.check_out()
168             elif choice == 5:
169                 hotel.show_history()
170             elif choice == 6:
171                 print("Exiting system. Goodbye!")
172                 break
173             else:
174                 print("Invalid choice. Please enter a number from 1 to 6.")
175
176
177 > if __name__ == "__main__":
178     main()
179

```

```
2. Book Room
3. Check-In
4. Check-Out & Bill
5. Booking History
6. Exit
Choose an option (1-6): 1
Room Availability
Room 101 | Price: $100.0 | Status: Available
Room 102 | Price: $150.0 | Status: Booked
Room 103 | Price: $200.0 | Status: Available

HOTEL SYSTEM MENU
1. View Rooms
2. Book Room
3. Check-In
4. Check-Out & Bill
5. Booking History
6. Exit
Choose an option (1-6):
```